NVFP4

Quantization of Mixtral 8 x 7b - Another Weekend Nvidia DGX Project!

An explanation and demonstration of LLM quantization using the Mixtral 8 x 7b MoE model, with a bit of maths, benchmarks and an ending that will require a sequel...

This weekend's hobby project is something I have been keen to try for a while, exploiting one of the bleeding-edge features of the GB10B chip, NVFP4, with the 'bleeding-edge' bit ultimately coming back to bite me when trying to test the results.

The project may sound complex on the face of it, but is actually relatively simple to execute and explain;

Modern LLMs are built on the transformer architecture and consist of the following layers:

1. Embedding layers - which convert text tokens into numerical vectors.

2. Attention mechanisms - which allow the model to focus on relevant parts of the input.

3. Feed-forward networks - which process and transform these representations.

4. Output layers - which predict the next token.

Each of these components contains millions or billions of weight parameters. For example, Mixtral 8 x 7B contains approximately 46.7 billion parameters across its 8 expert networks. These parameters are stored as floating-point numbers and most modern LLMs, including Mixtral 8x7B, are distributed in BF16 format.

Each parameter (16 bits) occupies 2 bytes of memory. For a 46.7 billion parameter model like Mixtral, 46.7 billion × 2 bytes = 93.4 GB. This explains why Mixtral requires approximately 87-94GB of

memory to load — and that's before accounting for the key-value cache needed during inference.

This presents a practical problem. Most modern GPUs have 24-80GB of memory, and even high-end workstations struggle to run these models efficiently. Even the DGX Spark, a purpose-built edge device, has 128GB, so loading Mixtral as distributed is really pushing the limits. (Side muse... I can connect 2 Sparks together using a special QSFP cable to get 256GB... Surely I can justify this? maybe I can ask Mrs Dowling for another DGX Spark for Christmas?)

The solution (other than buying more hardware) is quantization; the process of reducing the precision of model weights from higher-bit representations (like BF16) to lower-bit representations (like INT4 of NVFP4). The goal is to dramatically reduce memory requirements while minimizing the impact on model quality.

Linear Quantization

In its simplest form, linear quantization maps continuous floating-point values to a discrete set of values that can be represented with fewer bits. The simplest form of quantization maps floating-point values to integers using a linear transformation:

q = round (x / scale)

Where:

q is the resulting quantized integer

x is the original floating-point value

scale determines the step size between quantized values (see below!)

For example, for 8-bit symmetric quantization we have 255 possible values. Q model might have weights in the range (-1.0, 1.0) that we want to quantize to 8-bit signed integers (-127, 127) to fit in our 255 8-bit range.

To calculate the scale value:

scale = (max_val - min_val) / (255 - 1)

scale = (1.0 - (-1.0)) / 254 = 0.00784

Using this, to quantize a value of 0.5:

q = round(0.5 / 0.00784) = round(63.78) = 64

if we dequantize back:

64 × 0.00784 = 0.50176

The quantization error is therefore (0.5 - 0.50176) = 0.00176

In other words, for a loss of only 0.00176, we have halved* the memory required to load the model. For most inference tasks, this loss of precision is worth the trade off... remember even this level of

quantization would bring a large model in reach of most modern consumer grade gpus.

4-bit Quantization - The Challenge!

With only 16 possible values, 4-bit quantization is more aggressive. Using signed 4-bit (-8 to +7), the same range [-1.0, 1.0] now has:

scale = 1.0 / 7 ≈ 0.143

Therefore, for the same example weight of 0.5:

q = round(0.5 / 0.143) = round(3.5) = 4

dequantized = 4 × 0.143 = 0.572

The error increases to 0.5 - 0.572 = 0.072. This is an order of magnitude larger than the 8-bit quantization above, but still acceptable for most inference tasks, and theoretically reduces the size of the model by 3-4x.

The Project!

Mixtral 8x7B Instruct is a Moe model that (at 87GB) is large enough to demonstrate quantization benefits whilst just squeezing onto a SINGLE DGX Spark! The 'Mixture of Experts' architecture provides an interesting test case.

The MoE architecture means only 2 of the 8 experts are active for any given token, making the effective parameter count during inference closer to 13B, however, all 46.7B parameters must still be loaded into memory.

The first step was to download the model from Huggingface and load into memory.

Benchmarking - Performance

Taking initial speed performance benchmarks is straightforward, using a python script to measure inference across 5 different prompts.

```
prompts = [
    "Explain quantum computing in simple terms.",
    "Write a Python function to implement binary search.",
    "What are the main causes of climate change?",
    "Describe the process of photosynthesis step by step.",
    "Write a short story about a robot learning to paint."
]

# For each prompt, run 3 times and measure:
for prompt in prompts:
    for run in range(3):
        start_time = ...
```

Each prompt generates exactly 100 new tokens using greedy decoding (no sampling) for reproducibility. We run each prompt 3 times and average the results to account for variance (in hindsight, it would have been better to run more comprehensive benchmarks, but that doesn't matter as you will see...)

Benchmarking - Quality (Perplexity)

The perplexity benchmark evaluates Mixtral's ability to a sequence of words during inference. A lower score means the model has assigned higher probability to the next token, theoretically indicating how well the model 'understands' the prompt. This is a key test for our quantization; we want to see gains in speed and size reduction, without increasing the perplexity score.

Again, Huggingface provides a script which loads the WikiText-2 test set from the datasets library. It performs the following steps;

1. First it concatenates all the text into a single string and tokenizes it.

2. For each window (size 2048 tokens), extract a chunk of input tokens.

3. Predict the likelihood of the next tokens, and calculate the prediction errors (or negative log-likelihood -NLL).

4. Finally, calculate the average of all the prediction errors, and then use that number to calculate the perplexity (more detailed explanation here https://huggingface.co/docs/transformers/perplexity).

Quantization

The NVFP4 quantization process uses the TensorRT Model Optimizer from Nvidia's TensorRT library;

```
import modelopt.torch.quantization as mtq
from modelopt.torch.export import export_hf_checkpoint

calibration_data = load_calibration_samples(tokenizer,
                                            num_samples=512)

def forward_loop(model):
    model.eval()
    with torch.no_grad():
        for batch in calibration_data:
            model(input_ids=batch["input_ids"],
                  attention_mask=batch["attention_mask"])
```

*Note the process used TensorRT is more complex than the linear quantization examples given above. For a proper explanation, skip to 'The Runtime Phase' here https://docs.nvidia.com/deeplearning/tensorrt/latest/architecture/how-trt-works.html

Monitoring the project, Nvitop showed that I was properly stretching the DGX Spark;

The little DGX was definitely feeling the heat. Samples at various time using the infra-red thermometer from Mrs Dowling's Ooni pizza oven showed that it occasionally hit 61.c surface temp, although generally hovered around 51.c

The Results (Partial)

This is the frustrating bit...

Results Summary:

Size: BF16 Baseline = 87GB, NVFP4 Quantized = 24.82GB

Compression Ratio: 3.5x

Reduction: 71%

Quantization Time: 5h 46mins

Tokens/sec: BF16 = 4.05, NVFP4 = ?

Perplexity: BF16 = 3.7, NVFP4 = ?

Quantization worked! The output model reduced in size broadly in line with expectations. But when coming to load the quantized model into memory the dependency errors started...

30 minutes of searching, finding nothing, and hacking around could have been saved as Gemini promptly explained

"NVFP4 quantization produces weights in a packed format that requires specialized CUDA kernels to unpack and execute. These packed weights cannot be loaded by standard HuggingFace transformers, they require inference engines with native FP4 kernel support."

TensorRT itself provided this error:

WARNING: Detected NVIDIA GB10 GPU, which may not yet be supported in this version of the container

Thanks Nvidia! ship expensive hardware and sort the dependencies later... To be fair Nvidia are not the only ones... Pytorch only added to the inevitable mothballing.

Found GPU0 NVIDIA GB10 which is of cuda capability 12.1.

Minimum and Maximum cuda capability supported by this version of PyTorch is (8.0) - (12.0)

Conclusion

So that is where I leave it, waiting for updates that will allow me to test and see whether the process created any meaningful benefit. One small contribution, I was able to upload the quantized model to huggingface and share with the community, surprisingly appearing to be the first NVFP4 quantized version of Mixtral.

https://huggingface.co/josephdowling10/Mixtral-8x7B-Instruct-v0.1-NVFP4